

WebSocket Chat Implementation Guide

This guide provides detailed instructions on how to implement and use the real-time chat functionality in the Real Estate API.

Overview

The application uses Socket.io for real-time bidirectional communication between clients and the server. The chat system allows buyers and sellers to communicate about properties in real-time.

Server-Side Implementation

Chat Gateway

The `ChatGateway` handles all WebSocket connections and events. It's located in `src/websocket/chat.gateway.ts`.

Key Features:

- Connection/disconnection handling
- Chat room management
- Real-time message broadcasting
- Typing indicators
- Message persistence

WebSocket Events

1. Connection Events

Client Connected

```
handleConnection(client: Socket) {
  console.log(`Client connected: ${client.id}`);
}
```

Client Disconnected

```
handleDisconnect(client: Socket) {
  console.log(`Client disconnected: ${client.id}`);
}
```

2. Join Chat Room

Event: `join_chat`

Client Request:

```
socket.emit('join_chat', {
  chatId: 'chat-uuid-here',
  userId: 'user-uuid-here'
});
```

Server Response:

```
socket.on('joined_chat', (data) => {
  console.log('Joined chat:', data.chatId);
});
```

3. Leave Chat Room

Event: `leave_chat`

Client Request:

```
socket.emit('leave_chat', {
  chatId: 'chat-uuid-here',
  userId: 'user-uuid-here'
});
```

Server Response:

```
socket.on('left_chat', (data) => {
  console.log('Left chat:', data.chatId);
});
```

4. Send Message

Event: `send_message`

Client Request:

```
socket.emit('send_message', {
  content: 'Hello, is this property still available?',
  chatId: 'chat-uuid-here',
  senderId: 'user-uuid-here'
});
```

Server Response:

```
// Confirmation to sender
socket.on('message_sent', (message) => {
  console.log('Message sent:', message);
});

// Broadcast to all users in the chat room
socket.on('receive_message', (message) => {
  console.log('New message:', message);
  // message object structure:
  // {
  //   id: 'message-uuid',
  //   content: 'message text',
  //   senderId: 'user-uuid',
  //   chatId: 'chat-uuid',
  //   createdAt: '2024-01-01T00:00:00.000Z',
  //   updatedAt: '2024-01-01T00:00:00.000Z'
  // }
});
```

5. Typing Indicator

Event: `typing`

Client Request:

```
// When user starts typing
socket.emit('typing', {
  chatId: 'chat-uuid-here',
  userId: 'user-uuid-here',
  isTyping: true
});

// When user stops typing
socket.emit('typing', {
  chatId: 'chat-uuid-here',
  userId: 'user-uuid-here',
  isTyping: false
});
```

Server Response:

```

socket.on('user_typing', (data) => {
  if (data.isTyping) {
    console.log(`User ${data.userId} is typing...`);
  } else {
    console.log(`User ${data.userId} stopped typing`);
  }
});

```

Client-Side Implementation

1. HTML/JavaScript Client

```

<!DOCTYPE html>
<html>
<head>
  <title>Real Estate Chat</title>
  <script src="https://cdn.socket.io/4.6.0/socket.io.min.js"></script>
</head>
<body>
  <div id="chat-container">
    <div id="messages"></div>
    <input type="text" id="message-input" placeholder="Type a message...">
    <button onclick="sendMessage()">Send</button>
  </div>

  <script>
    // Connect to WebSocket server
    const socket = io('http://localhost:3000');

    const chatId = 'your-chat-id';
    const userId = 'your-user-id';

    // Join chat room on connection
    socket.on('connect', () => {
      console.log('Connected to server');
      socket.emit('join_chat', { chatId, userId });
    });

    // Handle joining chat room
    socket.on('joined_chat', (data) => {
      console.log('Joined chat:', data.chatId);
    });

    // Receive messages
    socket.on('receive_message', (message) => {
      displayMessage(message);
    });

    // Handle typing indicators
    socket.on('user_typing', (data) => {
      showTypingIndicator(data.userId, data.isTyping);
    });

    // Send message function
    function sendMessage() {
      const input = document.getElementById('message-input');
      const content = input.value.trim();

      if (content) {
        socket.emit('send_message', {
          content,
          chatId,
          senderId: userId
        });
      }
    }
  </script>

```

```

    });
    input.value = '';
  }
}

// Display message in UI
function displayMessage(message) {
  const messagesDiv = document.getElementById('messages');
  const messageElement = document.createElement('div');
  messageElement.textContent = `${message.senderId}: ${message.content}`;
  messagesDiv.appendChild(messageElement);
}

// Show typing indicator
function showTypingIndicator(userId, isTyping) {
  // Implementation depends on your UI
  console.log(`User ${userId} typing:`, isTyping);
}

// Send typing indicator
const messageInput = document.getElementById('message-input');
let typingTimeout;

messageInput.addEventListener('input', () => {
  socket.emit('typing', { chatId, userId, isTyping: true });

  clearTimeout(typingTimeout);
  typingTimeout = setTimeout(() => {
    socket.emit('typing', { chatId, userId, isTyping: false });
  }, 1000);
});

// Cleanup on page unload
window.addEventListener('beforeunload', () => {
  socket.emit('leave_chat', { chatId, userId });
});
</script>
</body>
</html>

```

2. React Client Example

```

import { useEffect, useState } from 'react';
import io from 'socket.io-client';

function ChatComponent({ chatId, userId }) {
  const [socket, setSocket] = useState(null);
  const [messages, setMessages] = useState([]);
  const [inputMessage, setInputMessage] = useState('');
  const [typingUsers, setTypingUsers] = useState(new Set());

  useEffect(() => {
    // Initialize socket connection
    const newSocket = io('http://localhost:3000');
    setSocket(newSocket);

    // Join chat room
    newSocket.on('connect', () => {
      newSocket.emit('join_chat', { chatId, userId });
    });

    // Handle incoming messages
    newSocket.on('receive_message', (message) => {
      setMessages(prev => [...prev, message]);
    });
  });
}

```

```

// Handle typing indicators
newSocket.on('user_typing', (data) => {
  setTypingUsers(prev => {
    const newSet = new Set(prev);
    if (data.isTyping) {
      newSet.add(data.userId);
    } else {
      newSet.delete(data.userId);
    }
    return newSet;
  });
});

// Cleanup
return () => {
  newSocket.emit('leave_chat', { chatId, userId });
  newSocket.close();
};
}, [chatId, userId]);

const sendMessage = () => {
  if (socket && inputMessage.trim()) {
    socket.emit('send_message', {
      content: inputMessage,
      chatId,
      senderId: userId
    });
    setInputMessage('');
  }
};

const handleInputChange = (e) => {
  setInputMessage(e.target.value);

  if (socket) {
    socket.emit('typing', { chatId, userId, isTyping: true });

    // Debounce typing indicator
    clearTimeout(window.typingTimeout);
    window.typingTimeout = setTimeout(() => {
      socket.emit('typing', { chatId, userId, isTyping: false });
    }, 1000);
  }
};

return (
  <div className="chat-container">
    <div className="messages">
      {messages.map((msg, index) => (
        <div key={index} className="message">
          <strong>{msg.senderId}</strong> {msg.content}
        </div>
      ))}
    </div>

    {typingUsers.size > 0 && (
      <div className="typing-indicator">
        Someone is typing...
      </div>
    )}

    <div className="input-container">
      <input
        type="text"

```

```
    value={inputMessage}
    onChange={handleInputChange}
    onKeyPress={(e) => e.key === 'Enter' && sendMessage()}
    placeholder="Type a message..."
  />
  <button onClick={sendMessage}>Send</button>
</div>
</div>
);
}

export default ChatComponent;
```

3. Node.js Client Example

```

const io = require('socket.io-client');

const socket = io('http://localhost:3000');

const chatId = 'your-chat-id';
const userId = 'your-user-id';

// Connect and join chat
socket.on('connect', () => {
  console.log('Connected to server');
  socket.emit('join_chat', { chatId, userId });
});

// Handle joining chat
socket.on('joined_chat', (data) => {
  console.log('Joined chat:', data.chatId);

  // Send a message
  setTimeout(() => {
    socket.emit('send_message', {
      content: 'Hello from Node.js client!',
      chatId,
      senderId: userId
    });
  }, 1000);
});

// Receive messages
socket.on('receive_message', (message) => {
  console.log('New message:', message);
});

// Handle message sent confirmation
socket.on('message_sent', (message) => {
  console.log('Message sent successfully:', message);
});

// Handle typing indicators
socket.on('user_typing', (data) => {
  if (data.isTyping) {
    console.log(`User ${data.userId} is typing...`);
  } else {
    console.log(`User ${data.userId} stopped typing`);
  }
});

// Handle disconnection
socket.on('disconnect', () => {
  console.log('Disconnected from server');
});

// Graceful shutdown
process.on('SIGINT', () => {
  socket.emit('leave_chat', { chatId, userId });
  socket.close();
  process.exit();
});

```

Testing WebSocket with Postman

While Postman is primarily for REST APIs, you can test WebSocket connections:

1. Open Postman
2. Create a new WebSocket request
3. Enter the URL: `ws://localhost:3000`

4. Connect to the WebSocket
5. Send events in JSON format:

```
{
  "event": "join_chat",
  "data": {
    "chatId": "chat-uuid",
    "userId": "user-uuid"
  }
}
```

Best Practices

1. Authentication

Consider adding authentication to WebSocket connections:

```
// In chat.gateway.ts
handleConnection(client: Socket) {
  const token = client.handshake.auth.token;
  // Verify JWT token
  // Store user info in client data
}
```

2. Error Handling

Always handle errors gracefully:

```
socket.on('connect_error', (error) => {
  console.error('Connection error:', error);
});

socket.on('error', (error) => {
  console.error('Socket error:', error);
});
```

3. Reconnection

Implement automatic reconnection:

```
const socket = io('http://localhost:3000', {
  reconnection: true,
  reconnectionDelay: 1000,
  reconnectionAttempts: 5
});
```

4. Room Management

Always leave rooms when done:

```
// Leave room before closing connection
window.addEventListener('beforeunload', () => {
  socket.emit('leave_chat', { chatId, userId });
});
```

5. Message Persistence

Messages are automatically persisted to the database. Retrieve message history using the REST API:

```
GET /chats/{chatId}/messages
```

Troubleshooting

Connection Issues

Problem: Cannot connect to WebSocket server

Solutions:

- Verify the server is running on the correct port
- Check CORS settings in the gateway
- Ensure firewall allows WebSocket connections

Message Not Received

Problem: Messages sent but not received by other users

Solutions:

- Verify both users have joined the same chat room
- Check chat ID matches on both clients
- Ensure the server is broadcasting to the correct room

Typing Indicator Not Working

Problem: Typing indicators not showing

Solutions:

- Ensure typing events are emitted with correct chat ID
- Verify the debounce timeout is working correctly
- Check that the receiving client is listening for `user_typing` events

Performance Considerations

1. **Message Rate Limiting:** Implement rate limiting to prevent spam
2. **Room Size:** Monitor the number of users per chat room
3. **Message History:** Load message history in batches
4. **Connection Pooling:** Use connection pooling for database operations
5. **Caching:** Cache frequently accessed chat data

Security Considerations

1. **Authentication:** Always verify user identity before allowing chat access
2. **Authorization:** Ensure users can only access chats they're part of
3. **Input Sanitization:** Sanitize all message content before storing
4. **Rate Limiting:** Implement rate limiting on message sending
5. **XSS Protection:** Escape HTML in message content when displaying

Future Enhancements

- File/image sharing in chat
- Message read receipts
- Message editing and deletion
- Push notifications for new messages
- Voice/video calling
- Group chats for multiple buyers/sellers
- Message search functionality
- Emoji reactions to messages

Support

For issues or questions about WebSocket implementation, please open an issue in the repository.